CATÁLOGO INTERATIVO 2.0

Entre Episódios.

Séries de televisão • API TVmaze

Rascunho de entrega

Este documento não está pronto para submissão. Faltam validação do aplicativo, publicação, dados do grupo, vídeo e screenshots reais.

Disciplina

[PREENCHER]

Turma

[PREENCHER]

Data real de entrega

[PREENCHER — prazo informado: 02/11/2026]

Integrantes — nomes completos de TODOS

[PREENCHER, um por linha]

Confirmar que séries/TVmaze não foram usados no Somativo de Flutter. Não reivindicar os bônus antes de testar a nuvem.

Seção 1 — Links

GitHub público: [INSERIR LINK CONFIRMADO DO REPOSITÓRIO PUBLICADO]

YouTube público: [INSERIR LINK DO VÍDEO GRAVADO E PUBLICADO]

Seção 2 — Tema e configuração

Tema proposto: catálogo de séries. API: https://www.tvmaze.com/api.

Baseline: persistência local com AsyncStorage; cadastro/login locais com hash de senha e sessão no SecureStore no Android/iOS. Na web, o adaptador usa armazenamento do navegador.

Bônus: integração preparada para Supabase Auth e banco PostgreSQL com RLS. **Só alterar a opção de entrega para “Local + Nuvem / Autenticação real” depois de configurar e comprovar.**

Configuração efetivamente usada no vídeo: [PREENCHER APÓS TESTAR].

Seção 3 — Autoavaliação RF01–RF10

O estado “Parcial” é conservador: existe implementação em código, mas a execução completa ainda não foi validada. Atualizar para Sim/Não/Parcial conforme a execução real, sem confundir existência de código com comprovação de funcionamento.

RF	Implementado	Caminho do arquivo principal
RF01	Parcial	`src/screens/CatalogScreen.tsx`
RF02	Parcial	`src/navigation/RootNavigator.tsx`
RF03	Parcial	`src/screens/DetailsScreen.tsx`
RF04	Parcial	`src/contexts/LibraryContext.tsx`
RF05	Parcial	`src/screens/LibraryScreen.tsx`
RF06	Parcial	`src/services/libraryStorage.ts`
RF07	Parcial	`src/contexts/AuthContext.tsx`
RF08	Parcial	`src/screens/SearchScreen.tsx`
RF09	Parcial	`src/hooks/useRequest.ts`
RF10	Parcial	`src/components/Button.tsx`

Seção 4 — Flutter × React Native

Texto técnico-base para o grupo adaptar à experiência real; não é um relato confirmado do Somativo de Flutter.

No Flutter, o Provider pode disponibilizar um estado a uma parte da árvore de widgets, enquanto no React Native usamos Context API e hooks para compartilhar sessão e biblioteca. A decisão de arquitetura é semelhante: estado usado por várias telas precisa ter uma fonte comum. Neste projeto, campos de formulário, filtro e busca continuam locais; favoritos e sessão ficam em Providers separados. O Context não elimina automaticamente renderizações: consumidores podem renderizar novamente quando seu valor muda.

Para o catálogo, a comparação é entre GridView.builder e FlatList com numColumns. As duas abordagens permitem trabalhar com listas extensas sem criar uma tela estática com todos os elementos de uma vez. No React Native, utilizamos keyExtractor baseado no ID, onEndReached e ListFooterComponent. A paginação e a deduplicação ficam separadas da apresentação.

No assincronismo, FutureBuilder permite representar estados de um Future no Flutter. Aqui, useEffect dispara a requisição, useState representa carregamento, erro e dados, e o cleanup cancela a operação. Não são mecanismos idênticos: o efeito também precisa de dependências estáveis e proteção contra respostas antigas.

Em ferramentas, Flutter oferece hot reload; no fluxo React Native usamos Fast Refresh, Metro e Expo. O Expo facilita a abertura do projeto quando o ambiente e o dispositivo são compatíveis. A avaliação de produtividade depende da experiência efetiva do grupo, da configuração e dos problemas encontrados, não apenas da quantidade de linhas.

Complementar antes da entrega: [QUAL TEMA/API FOI USADO NO FLUTTER? O QUE FOI MAIS FÁCIL OU DIFÍCIL NOS DOIS TRABALHOS? UM EXEMPLO CONCRETO DO GRUPO.]

Seção 5 — Explicação técnica

Tema: Persistência — AsyncStorage × SecureStore × nuvem.

Minuto planejado: 5:30 a 8:00. [CONFIRMAR O MINUTO NO VÍDEO PUBLICADO.]

Arquivos: `src/services/libraryStorage.ts`, `src/services/secureStorage.ts`, `src/services/localAuth.ts` e `src/contexts/LibraryContext.tsx`.

Seção 6 — Screenshots

Inserir seis capturas reais do aplicativo executado. Os quadros abaixo são espaços reservados, não screenshots.

Login

INSERIR CAPTURA REAL

Pendente de execução do aplicativo

Catálogo

INSERIR CAPTURA REAL

Pendente de execução do aplicativo

Detalhes

INSERIR CAPTURA REAL

Pendente de execução do aplicativo

Busca

INSERIR CAPTURA REAL

Pendente de execução do aplicativo

Favoritas

INSERIR CAPTURA REAL

Pendente de execução do aplicativo

Assistidas e notas

INSERIR CAPTURA REAL

Pendente de execução do aplicativo